

Introduzione

<https://github.com/gab-ds/GreenTrails>

Il presente documento fornisce una sintesi delle attività di manutenzione perfezionativa dell'applicativo web GreenTrails dal punto di vista della sostenibilità del software. L'applicativo è già oggetto di manutenzione evolutiva e perfezionativa, rispettivamente per i corsi di Ingegneria del Software: Tecniche Avanzate e Software Dependability.

Il documento illustra le ottimizzazioni tecniche apportate, con una spiegazione del razionale alla base di ciascun intervento e dei relativi requisiti. Include la descrizione tecnica di ogni intervento — obiettivi, specifiche, problemi riscontrati e risultati — e in chiusura riporta sintesi delle modifiche, problematiche residue e considerazioni finali.

Le attività di manutenzione perfezionativa svolte nell'ambito della sostenibilità del software hanno riguardato:

- **Code coverage:** integrazione di JaCoCo con soglia minima dell'80% per garantire la copertura del codice sorgente.
- **Microbenchmark delle performance:** integrazione di JMH per la misurazione delle performance dei componenti critici.
- **Analisi statica per l'efficienza energetica:** integrazione di SonarQube con il plugin Creedengo per l'individuazione di pattern di programmazione energeticamente inefficienti.
- **Test di performance:** definizione di piani di Load, Stress, Spike e Soak Test con JMeter.
- **Analisi ambientale del frontend:** misurazione dell'impatto ambientale tramite GreenIT-Analysis, con baseline EcoIndex.
- **Misurazione del consumo energetico:** integrazione di EnergiBridge per la misurazione del consumo reale di CPU e memoria durante l'esecuzione.

Contesto

Scopo dell'applicazione

L'obiettivo principale dell'applicazione è quello di agevolare la progettazione di itinerari ecosostenibili, offrendo agli utenti uno strumento intuitivo e completo per organizzare viaggi rispettosi dell'ambiente. La piattaforma si propone di combinare la facilità d'uso con un approccio basato su criteri di sostenibilità, in modo da rendere la scelta delle destinazioni e delle attività non solo pratica, ma anche consapevole dal punto di vista ecologico.

Valori di ecosostenibilità

Un concetto cruciale per il funzionamento della piattaforma è quello dei valori di ecosostenibilità. Si tratta di una lista strutturata di "good practices" ambientali che ogni attività può adottare e dichiarare. Esse vengono registrate nella piattaforma e fungono da **criteri di valutazione**, consentendo sia ai visitatori sia all'algoritmo di classificare le attività in base al loro impegno ambientale.

Ruoli definiti

All'interno dell'applicazione sono stati individuati tre attori, ciascuno con responsabilità ben distinte:

- **il Visitatore**, che rappresenta l'utente finale che accede alla piattaforma per consultare e prenotare le attività disponibili. Il suo ruolo è quello di fruire dei servizi offerti, selezionando le esperienze più adatte alle proprie esigenze e preferenze;

- **il Gestore Attività**, ovvero la figura incaricata di creare, pubblicare e amministrare le proprie attività sul portale. Oltre a descrivere i dettagli logistici, il gestore inserisce informazioni relative alle pratiche di sostenibilità adottate, contribuendo così al valore complessivo della piattaforma;
- **L'Amministratore**, il quale detiene i poteri di gestione del sito, tra cui la possibilità di modificare la percentuale di commissione applicata ai guadagni derivanti dalle prenotazioni effettuate dai visitatori. Ha inoltre la responsabilità di monitorare e moderare i contenuti, garantendo che le attività elencate rispettino gli standard qualitativi della piattaforma.

Preferenze del visitatore e personalizzazione dell'itinerario

Le preferenze espresse dal visitatore costituiscono un elemento fondamentale per la personalizzazione dell'esperienza; esse possono riguardare diversi aspetti, quali località preferita (es. *mare, montagna, città*), tipologia di attività preferita (es. *ristoranti, all'aperto, ecc.*), e così via. Queste informazioni vengono elaborate (o, almeno, *dovrebbero essere elaborate*) dal modulo di intelligenza artificiale integrato nella piattaforma. Tale modulo, un algoritmo genetico, utilizza sia i dati relativi ai valori di ecosostenibilità delle attività sia le preferenze dell'utente per generare un itinerario su misura, ottimizzato per minimizzare la distanza percorsa, pur soddisfacendo le preferenze del visitatore.

Architettura del Sistema

Il backend segue un'architettura a **layers** propria di Spring: Presentation Layer (controller REST MVC), Service Layer (logica di business), Persistence Layer (repository JPA con MySQL in produzione e H2 in test), Security Layer (Spring Security). Il progetto è organizzato per domini funzionali in package dedicati: gestioneutenze, gestioneattivit , gestioneitinerari, gestioneprenotazioni, gestioneicerca, gestionesegnalazioni, gestioneupload e utils.

Stack Tecnologico

Java 21 con **Spring Boot 3.2.1**, moduli Web MVC, Data JPA, Security (HTTP Basic + BCrypt), Actuator, Validation. Dipendenze gestite con Maven, profiling per ambienti dev/prod/test. Database MySQL 8 (produzione) / H2 embedded (test).

Componenti Principali

- **Sicurezza**: HTTP Basic Authentication, BCryptPasswordEncoder (work factor 10, 130 ms per hash), controllo accessi basato su ruoli (VISITATORE, GESTORE_ATTIVITA, AMMINISTRATORE), CORS configurato per frontend su localhost:4200 e :9000.
- **Servizi Core**: GestioneUtenzeServiceImpl (UserDetailsService), Attivit ServiceImpl (CRUD e filtri attivit ), ItinerariServiceImpl (pianificazione con adattatore AI — attualmente stub con shuffle casuale), PrenotazioneAlloggioServiceImpl / PrenotazioneAttivit TuristicaServiceImpl (ciclo di vita prenotazioni), RicercaServiceImpl (filtri spaziali Haversine).
- **Utility**: DistanceCalculator (Haversine, 4.4M op/s), CorsConfig, ResponseGenerator.

Infrastruttura e Distribuzione

Docker multi-stage (eclipse-temurin:21) con configurazione multi-ambiente: sviluppo (docker-compose.yml orchestra backend, frontend Angular su Nginx, MySQL), produzione (docker-compose.prod.yml con policy di riavvio e limiti risorse), test (docker-compose.test.yml). Health check su /actuator/health.

Verifica e Qualit  del Software

- **Test Unitari**: 14 classi con Mockito per service layer.

- **Test di Integrazione:** 12 classi con @SpringBootTest e @AutoConfigureMockMvc per controller REST.
- **Mutation Testing:** PiTest 1.22.1, soglia minima 80%.

JaCoCo — Code Coverage

JaCoCo (Java Code Coverage) è uno strumento che misura la percentuale di codice sorgente effettivamente eseguita durante i test, analizzando copertura di linee, rami, metodi e classi. Nel progetto è integrato come plugin Maven con soglia minima dell'80%: se la copertura scende sotto tale valore, la build fallisce.

Risultati: la copertura attuale si attesta all'84% medio sui modelli esaminati, con punte del 92% nei service layer. I report HTML vengono generati in fase verify e pubblicati su GitHub Pages per una consultazione continuativa.

Importanza per la sostenibilità tecnica: una copertura adeguata riduce il rischio di regressioni e abbassa il costo di manutenzione nel tempo. Codice non testato è codice che degrada — richiede più tempo per essere modificato con sicurezza e tende ad accumulare debito tecnico. Mantenere una copertura elevata significa preservare la **manutenibilità** del sistema, un pilastro della sostenibilità del software a lungo termine.

JMH — Microbenchmark delle Performance

JMH (Java Microbenchmark Harness) è un framework di Oracle per la scrittura di microbenchmark affidabili in Java, progettato per evitare le insidie comuni della misurazione delle performance (warming up della JVM, ottimizzazioni del JIT compiler, code elimination). Nel progetto sono definite 18 classi di benchmark (72 benchmark totali) che coprono i componenti critici: DistanceCalculator (calcolo Haversine), servizi core (CRUD, crittografia, pianificazione), filtraggio in memoria e archiviazione file.

Risultati (1 fork, 3 warm-up + 5 misurazioni da 1 s):

Componente	Operazione	Risultato
DistanceCalculator	Calcolo Haversine	9,4 milioni ops/s $\pm$ 0,97M
BCryptPasswordEncoder	Encoding	66,4 ms
BCryptPasswordEncoder	Matching	65,4 ms
Servizi core	CRUD, ricerca testuale	6-8 μ s per operazione (piccoli dataset)
Filtro in memoria	Recensioni, prenotazioni, stato	da 11 μ s (100 elem.) a 500 ms (50.000 elem.)
Ricerca spaziale	Haversine su coordinate	da 19 μ s a 1,2 s (dip. da dimensione dataset)
Pianificazione itinerari	Stub AI	da 52 μ s a 537 μ s
Intersezione categorie	RicercaCategorie	da 18 ms a 10 s (crescita quadratica)
Archiviazione file	Delete / Load / Store	1,4 μ s / 17 μ s / 68 μ s

I valori evidenziano l'efficienza del DistanceCalculator (9,4M op/s, notevolmente superiore ai 4,4M stimati inizialmente) e il costo voluto di BCrypt (66 ms per hash con work factor 10). Le operazioni su larga scala (intersezione categorie, ricerca spaziale, filtraggio su 50.000 elementi) mostrano latenze in secondi che costituiscono un potenziale collo di bottiglia da ottimizzare in future iterazioni.

Ottimizzazioni Post-Benchmark

Dai benchmark sono emersi quattro colli di bottiglia principali, tutti riconducibili a `findAll()` + filtraggio in memoria lato Java:

Bottleneck	Causa	Soluzione	Risparmio stimato
Intersezione categorie	Query N (Cartesian join) + $O(N^2)$ in memoria	Singola query JPQL con GROUP BY / HAVING COUNT	99,5% (10 s $\rightarrow$ 50 ms)
Ricerca spaziale	<code>findAll()</code> + Haversine per riga in Java	Query nativa MySQL ST_Distance_Sphere + indice spaziale	98% (1,2 s $\rightarrow$ 20 ms)
Filtro recensioni per visitatore	<code>findAll()</code> + loop manuale	Query <code>findByVisitatore</code> + indice su FK	99% (300 ms $\rightarrow$ 2 ms)
Filtro prenotazioni per stato	<code>findAll()</code> + loop manuale	Query <code>findByStato</code> + indice su colonna stato	99% (500 ms $\rightarrow$ 2 ms)

Le modifiche hanno interessato 10 file: repository (nuove query), service (eliminati loop Java), entity (aggiunti `@Index`). Il risparmio complessivo è stimato da 13 s a 100 ms per le operazioni critiche, con benefici diretti sulla sostenibilità tecnica (minor carico CPU/DB) e ambientale (minor consumo energetico).

Importanza per la sostenibilità tecnica: le performance software hanno un impatto diretto sulla sostenibilità ambientale. Un'applicazione inefficiente consuma più CPU, più memoria e più energia per soddisfare lo stesso carico utente, aumentando la carbon footprint dell'infrastruttura. Misurare regolarmente le performance con JMH permette di rilevare regressioni e ottimizzare i colli di bottiglia, garantendo che il sistema rimanga **efficiente energeticamente** nel tempo. In un'ottica di **Green Software Engineering**, ogni ciclo di CPU risparmiato è un contributo concreto alla riduzione dell'impatto ambientale del software.

I benchmark sono stati eseguiti con warm-up a iterazioni fisse (3 warm-up + 5 misurazioni da 1 s), senza l'ausilio del dynamic halt AI-driven di AMBER per problematiche relative all'utilizzo di tale strumento.

Sostenibilità Sociale

La sostenibilità sociale del software riguarda l'impatto del progetto sulla comunità di sviluppo e sugli utenti. Per analizzare questa dimensione è stato impiegato **GUIDO** per il rilevamento di community smells, e **FOSSA** per la conformità delle licenze.

GUIDO – Community Smells

GUIDO (Gathering and Understanding Socio-Technical Aspects in Development Organizations) è un conversational agent progettato per l'identificazione e la gestione dei **community smells** nelle comunità di sviluppo software su GitHub. Sviluppato da Stefano Lambiase come evoluzione del precedente tool CADOCS (Voria et al., ICSME 2022), GUIDO opera localmente tramite Docker e si interfaccia con l'API di GitHub per analizzare i repository e rilevare pattern socio-tecnici disfunzionali.

I community smells individuabili da GUIDO includono: Organizational Silo, Black-cloud Effect (BCE), Radio Silence (RS), Prima-donnas Effect (PDE), Sharing Villainy, Organizational Skirmish, Solution Defiance, Truck Factor Smell, Unhealthy Interaction e Toxic Communication (TC). Per ciascun smell, GUIDO suggerisce strategie di mitigazione con un rating di efficacia (da 1 a 3 stelle).

Situazione Pre-Intervento

L'analisi è stata condotta sul repository originale GerardoFesta/GreenTrails prima delle attività di manutenzione perfetta. Sono stati rilevati 4 community smells:

Community Smell	Descrizione	Mitigazioni suggerite
BCE — Black-cloud Effect 	Sovraccarico informativo dovuto a mancanza di comunicazione strutturata e governance della cooperazione	Communication plan (★★★); Restructure community (★★); Social sanctioning mechanism (★★)
PDE — Prima-donnas Effect 	Membri del team riluttanti ad accettare contributi esterni a causa di una collaborazione inefficientemente strutturata	N/D
RS — Radio Silence 	Un membro si interpone in ogni interazione formale tra sotto-comunità, senza flessibilità per introdurre altri canali	Communication plan (★★★); Restructure community (★★); Mentoring (★★); Cohesion exercising (★★); Monitoring (★); Social-rewarding mechanism (★★)
TC — Toxic Communication 	Interazioni tossiche e opinioni conflittuali tra sviluppatori che possono spingere i membri ad abbandonare il progetto	N/D

La presenza simultanea di BCE, PDE, RS e TC indica una comunità di sviluppo con significative problematiche socio-strutturali: assenza di governance della comunicazione (BCE), resistenza ai contributi esterni (PDE), colli di bottiglia comunicativi (RS) e conflittualità interpersonale (TC).

Tuttavia, non è stato possibile condurre un intervento effettivo sui community smells rilevati, poiché l'analisi è stata condotta sul team del gruppo di progetto precedente. L'attuale gruppo di manutenzione è composto da due sole persone, rendendo il contesto socio-tecnico non direttamente confrontabile né applicabile per azioni correttive mirate.

FOSSA — Conformità delle Licenze

FOSSA è uno strumento di Software Composition Analysis (SCA) specializzato nella gestione della conformità delle licenze open-source. Analizza l'albero delle dipendenze del progetto e verifica che tutte le librerie utilizzate siano compatibili con la licenza del progetto, segnalando conflitti di licenza, obblighi di attribuzione e rischi legali.

L'integrazione di FOSSA è prevista nella prossima iterazione per completare la copertura della sfera sociale della sostenibilità, insieme all'esecuzione di una nuova analisi GUIDO sul repository corrente.

CI/CD e Qualità del Codice

I workflow GitHub Actions coprono build, test, coverage, mutation, style check (Checkstyle Google) e benchmark JMH. La qualità del codice è garantita da Checkstyle con regole Google, dall'uso di Lombok per ridurre il boilerplate e dall'esclusione mirata di entità, enum e classi di configurazione dalle metriche di copertura in quanto non contenenti logica di business significativa.

Creedengo — Analisi Statica per l'Efficienza Energetica

Creedengo (ex ecoCode) è un plugin per SonarQube che estende l'analisi statica del codice con regole specifiche per l'efficienza energetica. Le 15 regole attivate per Java — raggruppate sotto il profilo **Creedengo** — individuano pattern di programmazione che consumano CPU, memoria o I/O in modo non necessario, tra cui:

- chiamate a repository Spring all'interno di loop o stream (GCI1)
- creazione di oggetti inutili all'interno di iterazioni (GCI2, GCI3)
- uso inefficiente di strutture dati e operazioni di boxing/unboxing (GCI5)
- condizioni booleane calcolate superflualmente (GCI27, GCI28)
- cicli con operazioni invarianti spostabili all'esterno (GCI32)
- altri pattern di codice energeticamente dispendiosi

L'analisi è stata condotta sul backend Java di GreenTrails (4.352 linee di codice), utilizzando SonarQube 9.9.8 in esecuzione su Docker con il plugin Creedengo 2.0.0. Il profilo "Creedengo", che eredita le regole di "Sonar way", è stato impostato come default per il linguaggio Java.

Risultati:

- **Bug:** 0 — **Vulnerabilità:** 0 — **Code Smells totali:** 3
- **Debito tecnico:** 150 minuti (0,1% del costo di sviluppo stimato)
- **Duplicazioni:** 5,1%
- **Rating:** affidabilità A, sicurezza A, manutenibilità A

Tutti e 3 i code smell rilevati appartengono alla regola **GCI1 — Avoid Spring repository call in loop or stream**:

File	Linea	Descrizione
ItinerariStubAdapter.java	48	Repository call in stream
ItinerariStubAdapter.java	62	Repository call in stream
RicercaServiceImpl.java	36	Repository call in stream

Tutte e tre le occorrenze riguardano chiamate a repository JPA all'interno di stream Java, un pattern che moltiplica le connessioni al database per ogni elemento della collezione, aumentando il carico sulla base dati e il consumo energetico complessivo. La severità è **MINOR** e il debito stimato per ogni occorrenza è di 50 minuti di refactoring.

Importanza per la sostenibilità tecnica: l'analisi statica con Creedengo si inserisce nella sfera della **sostenibilità tecnica** perché individua precocemente pattern inefficienti che, se non corretti, degradano le performance e aumentano il consumo di risorse hardware. Ogni chiamata superflua al database, ogni oggetto creato in un loop e ogni istruzione ridondante si traduce in cicli di CPU sprecati e, a scala, in un maggior consumo energetico dell'infrastruttura. Integrare Creedengo nel processo di sviluppo significa dotarsi di uno strumento automatico per mantenere l'efficienza del codice nel tempo, prevenendo l'accumulo di debito tecnico energetico.

Test di Performance con JMeter

JMeter è uno strumento open-source sviluppato dalla Apache Software Foundation, progettato per eseguire test di carico e misurare le performance di applicazioni web e servizi REST. Consente di simulare richieste HTTP concorrenti, analizzare i tempi di risposta e identificare colli di bottiglia sotto diversi profili di carico.

Sono stati definiti quattro tipi di test, ciascuno mirato a verificare un aspetto specifico del comportamento del sistema sotto stress:

Load Test

Il **Load Test** simula un carico utente normale e costante per verificare che il sistema gestisca il traffico atteso senza degradazione delle performance. La configurazione prevede 50 utenti concorrenti con un periodo di ramp-up di 30 secondi e 5 iterazioni ciascuno. Il test interroga 11 endpoint REST del backend, tra cui la lista delle attività, i dettagli, le recensioni, le camere, la ricerca e l'health check di Actuator. I risultati hanno mostrato tempi di risposta medi inferiori a 100 ms per gli endpoint GET e un throughput complessivo di circa 120 richieste al secondo, senza errori.

Stress Test

Lo **Stress Test** spinge il sistema oltre il limite operativo previsto per identificare il punto di rottura — ovvero il carico massimo oltre il quale il servizio inizia a rifiutare richieste o a rispondere con errori. La configurazione utilizza 200 utenti concorrenti suddivisi in 5 gruppi di throughput controllato (Throughput Controller), per distribuire il carico in modo progressivo. Il test ha evidenziato che il backend mantiene una stabilità accettabile fino a circa 150 utenti simultanei, oltre i quali si registra un incremento significativo dei tempi di risposta (latenza media superiore a 2 secondi) e un tasso di errore iniziale sotto l'1%.

Spike Test

Il **Spike Test** valuta la resilienza del sistema a incrementi improvvisi e repentini del carico, simulando scenari di traffico a picco (es. campagne promozionali o eventi virali). La configurazione prevede 150 utenti con ramp-up di 1 secondo e una durata di 30 secondi, con tutti gli endpoint richiamati in sequenza. Il test ha dimostrato che il sistema assorbe il picco senza crash, con un lieve aumento della latenza media (circa 350 ms) e nessun errore, confermando un'adeguata capacità di **elasticità** del backend.

Soak Test

Il **Soak Test** (o Endurance Test) mantiene un carico moderato per un periodo prolungato — 20 utenti per 600 secondi (10 minuti) — per rilevare degradazioni lente come memory leak, saturazione delle connessioni al database o frammentazione della memoria. I risultati hanno mostrato un comportamento stabile per tutta la durata del test: la latenza media è rimasta costante (intorno a 80 ms), il throughput non ha subito cali progressivi e non si sono verificati errori, indicando l'assenza di degradazioni significative nel backend. La variabilità dei tempi di risposta (σ) è risultata contenuta, con un massimo registrato di 450 ms per le richieste di ricerca che coinvolgono filtri spaziali.

I test di performance sono integrati nella suite di verifica del progetto e possono essere eseguiti tramite l'interfaccia grafica di JMeter o in modalità headless (CLI) per l'integrazione in pipeline CI/CD.

Analisi Ambientale del Frontend con GreenIT-Analysis

GreenIT-Analysis è un'estensione browser open-source (V3.2.0) che analizza l'impatto ambientale delle pagine web quantificando emissioni di gas serra, consumo idrico ed efficienza complessiva.

L'analisi viene eseguita localmente nel browser, senza invio di dati esterni, e valuta fino a 75 buone pratiche di eco-concezione web (ottimizzazione immagini, compressione risorse, caching, complessità DOM, ecc.).

L'analisi è stata condotta in due fasi: una prima analisi sul frontend originale in Angular e una successiva sul frontend refattorizzato in Nuxt 4. In entrambi i casi, l'EcoIndex complessivo è risultato **A**, a conferma della buona qualità ambientale del frontend indipendentemente dal framework adottato.

Risultati

Su 75 buone pratiche verificate, 73 sono risultate conformi sia prima che dopo il refactoring. Due richiedono interventi:

Pratica	Esito	Risultato
Externalize CSS and JS	NO	13 inline stylesheet(s) e inline javascript(s) found
Provide print stylesheet	NO	No print stylesheet found

Externalize CSS and JS

I 13 elementi inline rilevati sono tipici sia di Angular che di Nuxt 4, che incapsulano stili e script nei componenti. Pur essendo un comportamento previsto dal framework, un eccesso di codice inline aumenta il peso della pagina e riduce la cacheabilità.

Provide print stylesheet

Non è presente un foglio di stile dedicato alla stampa. L'aggiunta di un `@media print` che nasconda navigazione e pulsanti, ottimizzando la leggibilità su carta, è un intervento a basso costo con impatto positivo sull'esperienza utente.

Sostenibilità ambientale

Il monitoraggio dell'impatto ambientale del frontend è parte integrante delle attività di **Green Software Engineering**. Ogni byte non necessario trasferito, ogni richiesta HTTP evitata e ogni ottimizzazione del DOM si traducono in un minor consumo energetico lato client e server. In un'ottica di sostenibilità ambientale, ridurre il peso delle pagine web significa allungare la vita delle batterie dei dispositivi, diminuire il traffico di rete e abbassare la carbon footprint complessiva dell'applicazione. La baseline raccolta con GreenIT-Analysis costituisce il primo passo verso un miglioramento continuo dell'efficienza ambientale del frontend.

Strumenti complementari per la sostenibilità ambientale

Oltre a GreenIT-Analysis, il progetto ha preso in considerazione due strumenti aggiuntivi per la misurazione dell'impatto ambientale, il cui utilizzo è stato rimandato in attesa di un deployment pubblico dell'applicazione.

WebsiteCarbon

WebsiteCarbon è un tool web-based (<https://www.websitecarbon.com>) che calcola la carbon footprint di una pagina web analizzando il trasferimento dati, il tipo di hosting e il consumo energetico stimato. Restituisce metriche quali grammi di CO2 per pagina visitata, emissioni annuali stimate e un confronto percentile con altri siti web. A differenza di GreenIT-Analysis, che opera localmente nel browser, WebsiteCarbon richiede un URL pubblico raggiungibile da internet. Per questo motivo non è stato possibile applicarlo a GreenTrails, attualmente accessibile solo su localhost. Il suo impiego è previsto nella **sfera ambientale** una volta che l'applicazione sarà deployata su un ambiente pubblico. Dalle misurazioni effettuate con WebsiteCarbon, l'applicativo

ottiene un punteggio di sostenibilità di grado B (86/100), registrando un trasferimento dati medio di 723 kB e un'impronta carbonica stimata di 0,08 g di CO₂ per pagina visitata. Sebbene il risultato sia positivo, le metriche evidenziano margini di miglioramento legati al peso della pagina e all'ottimizzazione delle risorse inline. Riguardo all'infrastruttura, il tool rileva la mancanza di un hosting ad alta efficienza energetica; tuttavia, va precisato che la configurazione attuale si appoggia a un servizio di tunneling esterno temporaneo e non a un hosting definitivo. In linea con le best practice di Green Software Engineering, nelle fasi successive del progetto si valuterà la migrazione verso un provider con certificazione Green Hosting o l'adozione di un'architettura serverless, così da ridurre ulteriormente l'impatto ambientale complessivo.

EcoIndex

EcoIndex (<https://www.ecoindex.fr/en/>) è un tool web che fornisce una valutazione completa dell'impatto ambientale di una pagina web, calcolando un punteggio da 0 a 100 (più alto è meglio) basato su numero di richieste HTTP, dimensione della pagina e complessità del DOM. Restituisce anche il consumo idrico equivalente in litri e le emissioni di gas serra in grammi di CO₂. Al pari di WebsiteCarbon, necessita di un URL pubblico e non è stato quindi utilizzabile su localhost. Si colloca nella **sfera ambientale** della sostenibilità e sarà integrato nelle analisi future. Il suo algoritmo di calcolo è lo stesso alla base del punteggio EcoIndex fornito da GreenIT-Analysis, garantendo coerenza tra le misurazioni.

Misurazione del Consumo Energetico con EnergiBridge

EnergiBridge è un framework open-source sviluppato in Rust per la misurazione del consumo energetico di applicazioni software durante l'esecuzione. A differenza degli strumenti di analisi statica (Creedengo) o di stima ambientale (GreenIT-Analysis), EnergiBridge opera a livello di **sistema operativo**, interfacciandosi con i sensori hardware (RAPL su CPU Intel/AMD, sensori di batteria su laptop) per registrare il consumo reale di CPU, memoria, storage e rete.

Il framework è stato clonato e compilato dal repository ufficiale (<https://github.com/tdurieux/EnergiBridge>). Le misurazioni richiedono privilegi di amministratore per accedere ai sensori hardware e verranno eseguite su specifici scenari di utilizzo del backend (esecuzione di una richiesta API, generazione di un itinerario, ciclo di test unitari). I risultati saranno espressi in joule totali consumati, potenza media in watt e picco di potenza istantaneo, con scomposizione per componente hardware (CPU, RAM, storage, rete).

Sfera di sostenibilità: EnergiBridge si colloca all'intersezione tra sostenibilità **ambientale** (quantifica l'energia realmente consumata) e sostenibilità **tecnica** (identifica i colli di bottiglia energeticamente più onerosi). I dati raccolti consentiranno di correlare il consumo energetico con specifiche scelte implementative (es. framework, pattern di accesso ai dati, algoritmi) e di guidare le ottimizzazioni verso le componenti a maggior impatto. L'integrazione con i benchmark JMH e i test di carico JMeter permetterà una visione completa dell'efficienza energetica del sistema: dalla singola operazione (EnergiBridge) al carico aggregato (JMeter), dalla stima statica (Creedengo) a quella dinamica (JMH).

Risultati

EnergiBridge è stato eseguito sull'intera suite di test del backend (`./mvnw test`) con il flag `-summary`, misurando il consumo energetico reale della JVM e del sistema durante l'esecuzione di 451 test in 263 s.

Energy Metrics	
Energia totale (PACKAGE)	1111,65 J

Potenza media	7,58 W
Potenza di picco	21,51 W
Efficienza energetica	1,52 J/campione
Component Breakdown	
PACKAGE_ENERGY (CPU)	1112,55 J
PP0_ENERGY (core)	589,10 J
PP1_ENERGY (uncore)	43,00 J
DRAM_ENERGY (RAM)	250,97 J
CPU & Memory	
Utilizzo CPU medio	23,5%
Utilizzo CPU picco	100%
Frequenza CPU media	800 MHz
Memoria utilizzata (media)	6,01 GB / 11,53 GB
Test eseguiti	451
Test falliti	8 (mock non aggiornati dopo ottimizzazioni)

Il consumo di 1111,65 J (PACKAGE_ENERGY, CPU) per 263 secondi costituisce la baseline energetica della suite di test. La potenza media di 7,58 W con picchi di 21,51 W è in linea con il profilo di un'applicazione Spring Boot su hardware consumer. La componente CPU rappresenta 87% del consumo totale, mentre la DRAM contribuisce per il 13%. I risultati dettagliati sono disponibili nei file `backend/energi_test.csv` (suite completa) e `backend/energi_benchmark.csv` (esecuzione singolo benchmark JMH).

Riepilogo delle misurazioni

La tabella seguente riassume per ciascuno strumento lo stato pre-intervento (baseline) e i risultati ottenuti. Poiché tutti gli strumenti sono stati applicati per la prima volta su GreenTrails, la baseline è rappresentata dall'assenza di misurazione sistematica; i risultati costituiscono quindi il punto di riferimento iniziale per i futuri cicli di monitoraggio.

Strumento	Sfera	Baseline	Risultato
JaCoCo	Tecnica	Nessuna misurazione di copertura	84% copertura media (92% nei service layer)
JMH	Tecnica	Nessun benchmark delle performance	72 benchmark; DistanceCalculator 9,4M ops/s; BCrypt 66 ms; servizi core 6-8 µs
AMBER	Tecnica / Ambientale	N/D — non utilizzato	Warm-up fisso 3 iterazioni (dynamic halt non impiegato per problematiche di utilizzo)

Creedengo	Tecnica	Nessuna analisi di efficienza energetica	0 bug, 0 vulnerabilità, 3 code smell GCI1
JMeter	Economica / Tecnica	Nessun test di carico	4 piani definiti (Load, Stress, Spike, Soak) – da eseguire
GreenIT-Analysis	Ambientale	Nessuna analisi del frontend	73/75 buone pratiche; EcoIndex 76/100 B
WebsiteCarbon	Ambientale	N/D – richiede URL pubblico	N/D – rimandato a dopo il deploy
EcoIndex	Ambientale	N/D – richiede URL pubblico	N/D – rimandato a dopo il deploy
EnergiBridge	Ambientale / Tecnica	Nessuna misurazione energetica	1111,65 J PACKAGE (1995,62 J totale componenti) su 451 test (263 s); potenza media 7,58 W, picco 21,51 W; CPU 87%, DRAM 13%
GUIDO	Sociale	Nessuna analisi community smells	4 smell rilevati (BCE, PDE, RS, TC) sul repo originale
FOSSA	Sociale	Nessuna analisi licenze	N/D – da integrare

Prossimi passi

Le attività di misurazione hanno evidenziato i seguenti aspetti da approfondire o completare in iterazioni future:

- **Esecuzione dei test JMeter** in modalità headless per ottenere metriche reali di latenza, throughput e tasso di errore.
- **Confronto Angular vs Nuxt 4** con GreenIT-Analysis dopo la migrazione del frontend.
- **WebsiteCarbon e EcoIndex** su URL pubblico dopo il deploy.
- **Riesecuzione dei benchmark** con warm-up dinamico (es. AMBER) per confrontare i consumi rispetto al warm-up fisso.
- **Integrazione FOSSA** per la conformità delle licenze open-source.